

АСИНХРОННЫЙ RUST

Глубокий инженерный справочник: от байтов сетевой карты до рантайма Tokio

*С исчерпывающими сносками к каждой функции,
построчным анатомическим анализом кода,
стейт-машинами компилятора и разбором структуры FuturesUnordered*

Полное оглавление

Как устроена эта книга	3
Глава 1. Синхронный мир и блокирующий ввод-вывод (I/O)	4
1.1. Что на самом деле происходит при обычном вызове функции?	4
1.2. Почему 500 потоков ОС убивают сервер: Context Switch Tax	4
Глава 2. Как железо и ядро ОС работают с сетью	6
2.1. Анатомия файлового дескриптора (File Descriptor)	6
2.2. Неблокирующий режим (O_NONBLOCK)	6
2.3. Механика системного вызова <code>epoll</code> в Linux	6
Глава 3. Потоки ОС против Асинхронных задач Tokio	8
3.1. Сравнительная инженерная таблица	8
3.2. Как работает кооперативность?	8
Глава 4. Фундамент Rust: Что такое Future на самом деле?	10
4.1. Разница между JS Promise и Rust Future	10
4.2. Трейт Future: метод <code>poll</code> и перечисление <code>Poll</code>	10
4.3. Автоматическая генерация State Machine компилятором <code>rustc</code>	10
Глава 5. Механика Waker и Context: Как разбудить задачу?	12
5.1. Что такое RawWaker и виртуальная таблица вызовов (VTable)	12
5.2. Пошаговая хроника пробуждения: от сетевого пакета до выполнения	12
Глава 6. Анатомия рантайма Tokio: Реактор и Планировщик	14
6.1. Архитектура Multi-Threaded Work-Stealing Scheduler	14
Глава 7. Стримы (Stream): Асинхронные потоки данных	15
7.1. Трейт Stream и модель Pull	15
Глава 8. Анатомия <code>buffer_unordered</code> : Внутреннее устройство	16
8.1. Построчный анатомический разбор	16
8.2. Как устроен <code>FuturesUnordered</code> внутри?	16
8.3. Механика Sub-Wakers: Почему опрос занимает $O(1)$	17
8.4. Почему <code>buffer_unordered(25)</code> делает Семафор избыточным?	17
Глава 9. Полный анатомический разбор воркера Uptime-сервиса	19
Глава 10. Энциклопедический словарь-справочник	21
1. Системный уровень (Ядро Linux, C-библиотеки и ОС)	21
2. Уровень архитектуры компилятора Rust	21
3. Уровень рантайма Tokio и алгоритмов	21

Как устроена эта книга

Каждая глава этой книги содержит исчерпывающее техническое погружение. Мы разберем не просто общие концепции, а **каждую команду, каждый системный вызов, каждую строку кода и каждый бит в памяти**.

Для максимального удобства весь материал снабжен специальными блоками:

- 💡 **Интуиция «на пальцах»** — понятная жизненная аналогия, объясняющая суть явления.
- 🔍 **Построчный анатомический разбор** — детальный анализ каждой строки исходного кода.
- ⚙️ **Уровень гика** — как это реализовано в кремнии процессора, регистрах и ядре Linux.
- ⚠️ **Грабли на проде** — где 90% разработчиков совершают критические ошибки, роняющие сервис.
- ⚡ **Фишка / Pro-Tip** — практические трюки для максимальной производительности в продакшене.
- **Сноски внизу страниц** — подробная расшифровка каждого термина, сисколла, типажа и флага прямо там, где он встретился в тексте.

—

Глава 1. Синхронный мир и блокирующий ввод-вывод (I/O)

1.1. Что на самом деле происходит при обычном вызове функции?

Рассмотрим простейший синхронный код:

```
let response = request::blocking::get("https://google.com");
```

Что происходит под капотом шаг за шагом?

1. Библиотека вызывает функцию С-библиотеки `libc getaddrinfo`¹. Поток блокируется на время DNS-резолвинга.
2. Создается сокет через системный вызов `socket(AF_INET, SOCK_STREAM, 0)`².
3. Вызывается `connect()`³. Поток засыпает на время выполнения тройного рукопожатия TCP.
4. Отправляется HTTP-запрос через `write()`⁴.
5. Вызывается `read()`⁵. **И вот здесь наступает ступор.**

Построчный анатомический разбор:

Когда вызван `read()`, а сервер в Калифорнии еще не сформировал ответ:

1. Ядро Linux проверяет кольцевой буфер приема сокета (Receive Buffer). Он пуст.
2. Ядро берет структуру текущего потока `task_struct`⁶ и меняет поле `state` со значения `TASK_RUNNING` на `TASK_INTERRUPTIBLE`.
3. Поток помещается в очередь ожидания сокета (`wait_queue_head_t`).
4. Ядро вызывает планировщик `schedule()`⁷, который снимает поток с ядра процессора.

Интуиция «на пальцах»:

Синхронный блокирующий вызов — это заказ пиццы по телефону, при котором вы не кладете трубку, не уходите в туалет, не пьете воду, а стоите и держите трубку у уха 40 минут, пока курьер едет к вашему дому.

1.2. Почему 500 потоков ОС убивают сервер: Context Switch Tax

Если запустить по одному системному потоку (`std::thread::spawn`) на каждый сайт, сервер столкнется с физическими ограничениями архитектуры x86-64:

¹`getaddrinfo()` — функция стандартной библиотеки C (`libc`), которая преобразует человекочитаемое доменное имя (например, `google.com`) в машинный IP-адрес через отправку DNS-запроса по протоколу UDP на порт 53 локального или заданного DNS-сервера.

²`socket(domain, type, protocol)` — системный вызов ядра ОС (`syscall`). Создает конечную точку сетевого соединения. `AF_INET` означает IPv4, `SOCK_STREAM` — надежный потоковый протокол с установлением соединения (TCP).

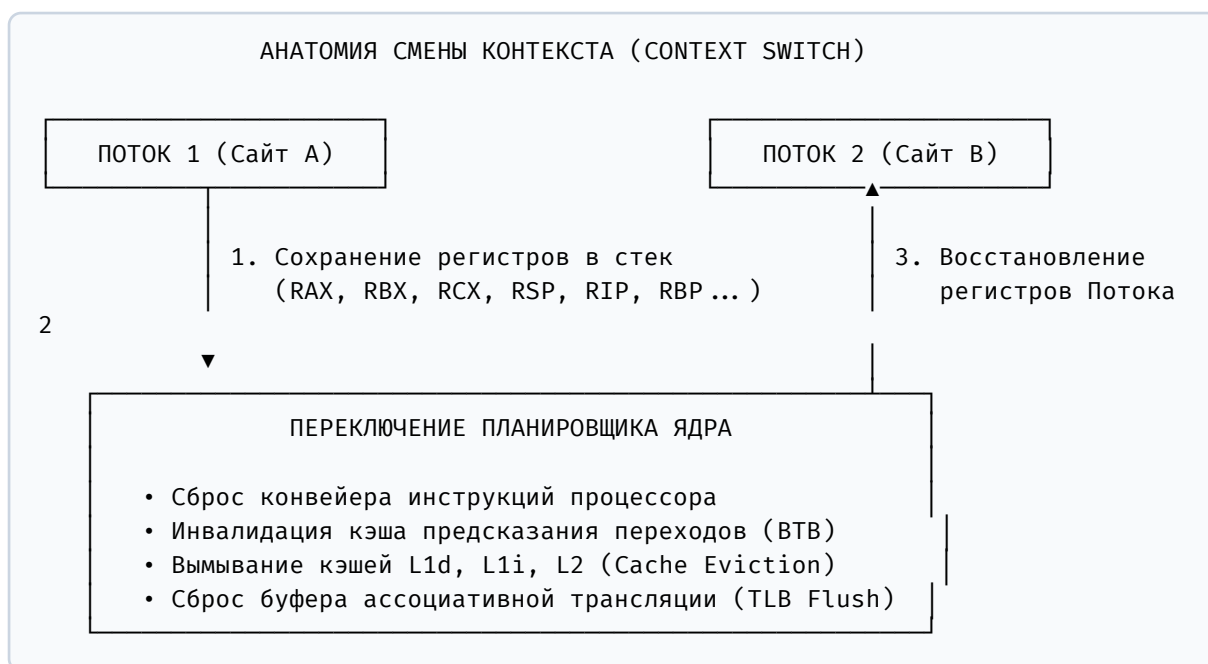
³`connect(fd, addr, addrlen)` — системный вызов, инициирующий 3-way handshake по протоколу TCP: клиент шлет пакет SYN, сервер отвечает SYN-ACK, клиент подтверждает ACK. До завершения этой процедуры поток усыпляется ядром.

⁴`write(fd, buf, count)` — системный вызов ядра, копирующий байты из буфера в пространстве пользователя (User Space) в передающий буфер сокета в памяти ядра (Kernel Space).

⁵`read(fd, buf, count)` — системный вызов, запрашивающий байты из принимающего буфера сокета ядра в буфер приложения.

⁶`task_struct` — главная структура описания процесса или потока внутри ядра Linux (определена в `<linux/sched.h>`). Содержит регистры, информацию о виртуальной памяти `mm_struct`, таблицу файлов `files_struct` и приоритеты.

⁷`schedule()` — внутренняя функция планировщика ядра Linux (CFS / EEVDF), которая выбирает следующий готовый к выполнению поток из дерева `rb_node` и передает ему управление процессором.



1. **Раздувание виртуальной памяти (VMA Bloat):** Каждый поток ОС резервирует стек фиксированного размера (по умолчанию в Linux — **8192 КБ / 8 МБ**, контролируется через `ulimit -s`). Для 10 000 потоков ядро выделит адресное пространство в **80 Гигабайт!**
2. **Промаш мимо кэша процессора (Cache Misses):** Кэш L1 процессора имеет объем всего 32–48 КБ на ядро. Поток 1 загружает свои данные в L1. Через 1 миллисекунду планировщик переключает ядро на Поток 2, который затирает данные Потока 1 своими данными. Когда управление возвращается Потoku 1, его данных в L1 уже нет — процессор вынужден лезть в медленную оперативную память RAM (задержка возрастает с 1 наносекунды до 60–100 наносекунд!).
3. **Предел дескрипторов и потоков:** Параметр ядра `/proc/sys/kernel/pid_max` по умолчанию ограничивает общее число процессов/потоков в системе числом 32 768.

⚠ Грабли на проде (Подводные камни и баги):

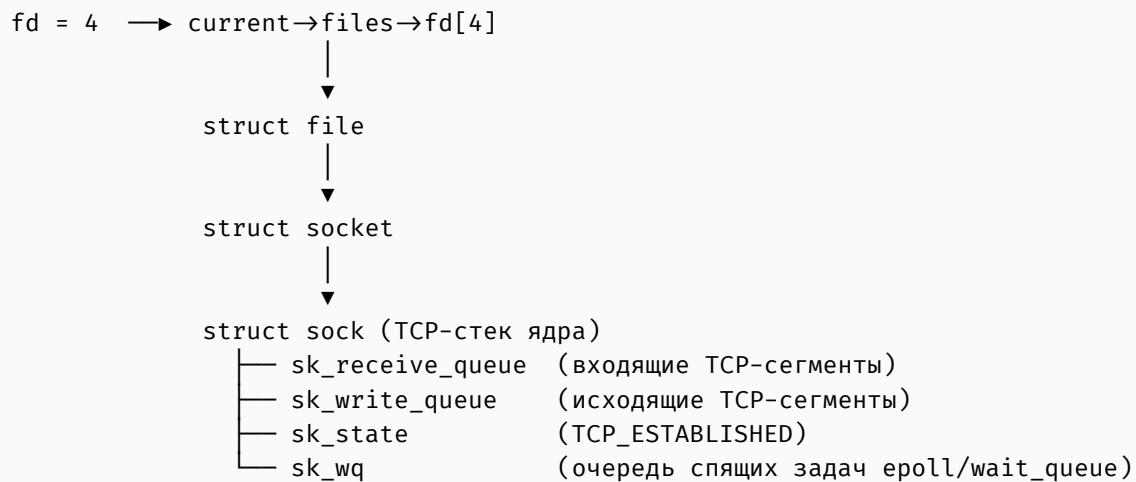
Никогда не используйте `std::thread::spawn` для I/O-bound задач (сеть, диск). Системные потоки предназначены исключительно для CPU-bound задач (тяжелый рендеринг видео, машинное обучение, криптография), где число потоков строго равно числу физических ядер процессора.

Глава 2. Как железо и ядро ОС работают с сетью

2.1. Анатомия файлового дескриптора (File Descriptor)

В ядре Linux файловый дескриптор — это всего лишь 32-битное целое число (`int fd`), которое является индексом в массиве таблицы файлов текущего процесса `current→files→fdt→fd[fd]`.

По этому индексу ядро достает структуру `struct file`, которая для сокета указывает на `struct socket` и лежащий под ней `struct sock`:



2.2. Неблокирующий режим (O_NONBLOCK)

Чтобы сокет не усыплял поток при отсутствии данных, мы переводим его в неблокирующий режим с помощью системного вызова `fcntl()`⁸:

```
int flags = fcntl(fd, F_GETFL, 0);
fcntl(fd, F_SETFL, flags | O_NONBLOCK);
```

Теперь поведение функции `read()` меняется кардинально:

- Если в `sk_receive_queue` есть байты: ядро копирует их в буфер и возвращает число прочитанных байт.
- Если в очереди пусто: ядро **НЕ усыпляет поток**, а мгновенно возвращает число `-1`, а в глобальную переменную ошибки `errno` записывает код `EWOULDBLOCK` (или `EAGAIN`)⁹.

2.3. Механика системного вызова `epoll` в Linux

Если опрашивать 10 000 сокетов через цикл `read()`, процессор будет загружен на 100% бесполезными проверками. Поэтому ядро Linux предоставляет подсистему **`epoll`** (Event Poll).

Она состоит из трех базовых сисколлов:

1. `epoll_create1(0)` → Создает контекст ядра (черно-белое дерево сокетов)

⁸`fcntl(fd, cmd, arg)` — системный вызов (File Control), позволяющий менять свойства открытого дескриптора. Флаг `O_NONBLOCK` переводит операции ввода-вывода в асинхронный/неблокирующий режим.

⁹`EAGAIN` / `EWOULDBLOCK` — стандартный POSIX код ошибки, означающий: «Ресурс временно недоступен. Повторите попытку позже».

2. `epoll_ctl(epfd, ADD, fd, EV)` → Добавляет сокет под наблюдение ядра
3. `epoll_wait(epfd, events, ...)` → Усыпляет поток до реального прихода пакетов

⚙️ Уровень гика (Ядро ОС, регистры CPU и структуры памяти):

Внутри ядра дескриптор `epoll` устроен как **Красно-черное дерево (Red-Black Tree)** для быстрого поиска сокетов по их номеру за $O(\log N)$ при добавлении/удалении, и **Связный список готовых событий (Ready List)**, куда прерывания ядра за $O(1)$ добавляют сокет, в которые пришли пакеты.

Когда физический кабель сетевой карты принимает электромагнитные импульсы:

1. Сетевой контроллер (NIC) через прямой доступ к памяти (**DMA** — Direct Memory Access)¹⁰ перекладывает байты пакета в кольцевой буфер оперативной памяти (Ring Buffer).
2. Сетевая карта отправляет процессору сигнал прерывания линии (**MSI-X / IRQ**)¹¹.
3. Ядро запускает обработчик пакета (SoftIRQ / NAPI poll)¹², парсит TCP-заголовок, находит нужный сокет по портам и IP.
4. Ядро кладет пакет в `sk_receive_queue` сокета и добавляет сокет в Ready List структуры `epoll`.
5. Поток, спавший в вызове `epoll_wait()`, просыпается и получает массив **только тех сокетов, где уже есть данные!** Сложность обработки — строго $O(1)$ относительно общего числа соединений.

¹⁰DMA (Direct Memory Access) — механизм, позволяющий сетевой карте писать принятые пакеты напрямую в оперативную память (RAM) хоста, вообще не нагружая центральный процессор.

¹¹IRQ (Interrupt Request) — сигнал аппаратного прерывания, по которому процессор мгновенно приостанавливает текущие вычисления и запускает обработчик прерываний ядра (Interrupt Handler / Top-half ISR).

¹²NAPI (New API) — гибридная схема подсистемы сети Linux, где при высокой нагрузке прерывания отключаются, а ядро переходит в режим периодического опроса очередей сетевой карты, защищая систему от лавины прерываний (Interrupt Storm).

Глава 3. Потоки ОС против Асинхронных задач Tokio

3.1. Сравнительная инженерная таблица

Параметр	Поток ОС (Kernel Thread)	Tokio Task (Корутина / Green Thread)
Структура данных	task_struct внутри ядра Linux	Обычная структура Rust в куче (Box<Task> / Header)
Размер стека	Фиксированный (от 2 до 8 МБ)	Динамический: равен размеру стейт-машины (от 64 байт до пары КБ)
Стоимость создания	Системный вызов clone(2) (15 микросекунд)	Обычная аллокация в куче malloc (20 наносекунд)
Стоимость переключения	Смена адресного пространства, сброс TLB, смена стека ядра (1–3 микросекунды)	Вызов функции по указателю poll() (2–5 наносекунд)
Лимит на машине	Обычно 10 000 – 30 000 потоков	Миллионы задач на 1 ГБ оперативной памяти
Вытеснение	Принудительное (по таймеру ядра ОС каждые 1–4 мс)	Кооперативное (задача сама отдает ход в точках .await)

3.2. Как работает кооперативность?

Взгляните на простейшую задачу:

```

async fn process_site(site: Site) {
    let raw_html = fetch_html(&site.url).await; // 👉 Точка yield (отдача хода)
    let parsed = parse_html(raw_html);          // 👉 Синхронная работа (держит поток)
    save_to_db(parsed).await;                    // 👉 Точка yield (отдача хода)
}

```

- Строка 2: Вызывается `fetch_html`. Если сокет не готов, задача возвращает `Poll::Pending`. Воркер Tokio немедленно бросает эту задачу и берет другую!
- Строка 3: `parse_html` — чистая работа процессора. В это время поток Tokio занят **только этой задачей**. Никакие другие задачи на этом воркере не запустятся, пока парсинг не закончится.
- Строка 4: Вызывается `save_to_db`. Задача снова отдает ход планировщику Tokio.

⚡ Фишка / Pro-Tip для продакшена:

Если внутри `async fn` вам необходимо выполнить тяжелую операцию на CPU (например, разархивировать zip-архив на 100 МБ или хешировать пароль через Argon2), **никогда не делайте это прямо в async потоке!** Оберните ее в:

```
let hash = tokio::task::spawn_blocking(move || {  
    argon2::hash_password(password)  
}).await.unwrap();
```

`spawn_blocking` отправляет вычисление в отдельный резервный пул тяжелых потоков ОС, не блокируя быстрые сетевые воркеры Tokio!

Глава 4. Фундамент Rust: Что такое `Future` на самом деле?

4.1. Разница между JS Promise и Rust Future

В мире веб-разработки (Node.js, Go, C#) распространена модель **Eager Evaluation (Жадное вычисление)**: как только вы вызвали функцию `fetch()`, фоновый поток уже побежал в сеть.

В Rust принята модель **Zero-Cost Abstraction / Lazy Evaluation (Абсолютная ленивость)**:

```
let my_future = pinger.ping(&site); // 🙅 В СЕТЬ НЕ УШЛО НИ ОДНОГО БАЙТА!
```

Переменная `my_future` — это просто кусок памяти на стеке, где записаны параметры функции. Пока вы не напишете `.await` или не передадите её в `tokio::spawn`, она является просто «мертвой» структурой данных.

4.2. Трейт `Future`: метод `poll` и перечисление `Poll`

Определение из ядра стандартной библиотеки `core::future::Future`:

```
pub trait Future {
    type Output;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) →
    Poll<Self::Output>;
}

pub enum Poll<T> {
    Ready(T),
    Pending,
}
```

Построчный анатомический разбор:

1. `type Output;` — ассоциированный тип, означающий, какой результат вернет футура, когда успешно завершится (например, `Result<Response, Error>`).
2. `self: Pin<&mut Self>` — мутабельная ссылка на саму футуру, «приколота» к конкретному адресу памяти через `Pin`¹³. Если переместить футуру в памяти, ее внутренние сырые указатели на локальные переменные станут невалидными (Use-After-Free).
3. `cx: &mut Context<'_>` — контекст выполнения, в котором хранится ссылка на `&Waker`.
4. `Poll<Self::Output>` — результат:
 - `Poll::Ready(val)` — вычисление завершено, забирай значение!
 - `Poll::Pending` — вычисление заблокировано в ожидании события, задача усыплена.

4.3. Автоматическая генерация State Machine компилятором `rustc`

Когда вы пишете:

¹³`Pin<P>` — фундаментальный тип-обертка в Rust. Он гарантирует, что лежащие под ним данные никогда не будут перемещены (memmove) по другому адресу в оперативной памяти. Это необходимо, так как компилятор генерирует самоссылающиеся структуры (указатели на локальные переменные внутри той же футуры).

```

async fn example(url: String) → String {
    let a = step_one(&url).await;
    let b = step_two(&a).await;
    b
}

```

Компилятор Rust буквально переписывает эту функцию в монолитный `enum`:

```

// ТО, ВО ЧТО ВАШ КОД ПРЕВРАЩАЕТСЯ ПОД КАПОТОМ
enum ExampleStateMachine {
    // Состояние 0: только создана
    State0 { url: String },

    // Состояние 1: ждем завершения step_one
    State1 {
        fut_step_one: StepOneFuture,
    },

    // Состояние 2: step_one завершился, локальная переменная 'a' сохранена!
    // Ждем завершения step_two
    State2 {
        a: String, // Значение 'a' переехало внутрь enum!
        fut_step_two: StepTwoFuture,
    },

    // Состояние 3: все завершено
    Done,
}

```

⚙️ Уровень гика (Ядро ОС, регистры CPU и структуры памяти):

Размер футуры в памяти равен размеру **самого большого состояния этого `enum`** плюс дискриминант тэга (1–8 байт). Все локальные переменные, которые должны жить между точками `.await`, упаковываются компилятором в поля этого `enum`. Память под переменные выделяется ровно один раз! Никаких динамических аллокаций в куче на каждом шаге `.await` не происходит.

Глава 5. Механика `Waker` и `Context`: Как разбудить задачу?

5.1. Что такое `RawWaker` и виртуальная таблица вызовов (`VTable`)

`Waker` спроектирован так, чтобы работать с максимальной производительностью без привязки к стандартной библиотеке (даже в `no_std` на микроконтроллерах).

Внутри `Waker` состоит из указателя на данные и виртуальной таблицы функций:

```
pub struct RawWaker {
    data: *const (),
    vtable: &'static RawWakerVTable,
}

pub struct RawWakerVTable {
    clone: unsafe fn(*const ()) → RawWaker,
    wake: unsafe fn(*const ()),           // 🖱️ Поглощает владение и будит
    wake_by_ref: unsafe fn(*const ()),    // 🖱️ Будит по ссылке
    drop: unsafe fn(*const ()),           // 🖱️ Освобождает память
}
```

Построчный анатомический разбор:

1. `data: *const ()` — сырой бестиповый указатель на структуру задачи в куче (`Arc<Task>`).
2. `vtable` — набор из 4 указателей на машинные инструкции функций.
3. Когда сетевой сокет получает пакет, `Tokio` берет этот `data`, приводит его к указателю на свою внутреннюю задачу `Task` и вызывает функцию `wake()`.

5.2. Пошаговая хроника пробуждения: от сетевого пакета до выполнения

Посмотрим, как выглядит жизненный цикл пробуждения в реальном времени с точностью до миллисекунд:

0.000 мс	Tokio опрашивает сокет: <code>read()</code> возвращает <code>EWOULDBLOCK</code> . Сокет сохраняет копию <code>Waker</code> в <code>sk_wq</code> .
0.001 мс	Футура возвращает <code>Poll::Pending</code> . Воркер Tokio идет делать другие дела.
...	ПРОСТОЙ 45 МИЛЛИСЕКУНД (ПАКЕТ ЛЕТИТ ЧЕРЕЗ ИНТЕРНЕТ) ...
45.000 мс	Физический чип сетевой карты ловит пакет 200 OK.
45.002 мс	Контроллер карты пишет пакет через DMA в память и генерирует прерывание IRQ.
45.005 мс	Ядро Linux обрабатывает прерывание и кладет пакет в сокет.
45.010 мс	Реактор Tokio выходит из <code>epoll_wait(epfd, ...)</code> .
45.012 мс	Реактор достает сохраненный <code>Waker</code> и вызывает <code>waker.wake()</code> .
45.015 мс	Функция <code>wake()</code> берет задачу и кладет ее в очередь <code>Run Queue</code> воркера.
45.018 мс	Воркер достает задачу из очереди и снова вызывает <code>future.poll(cx)</code> .

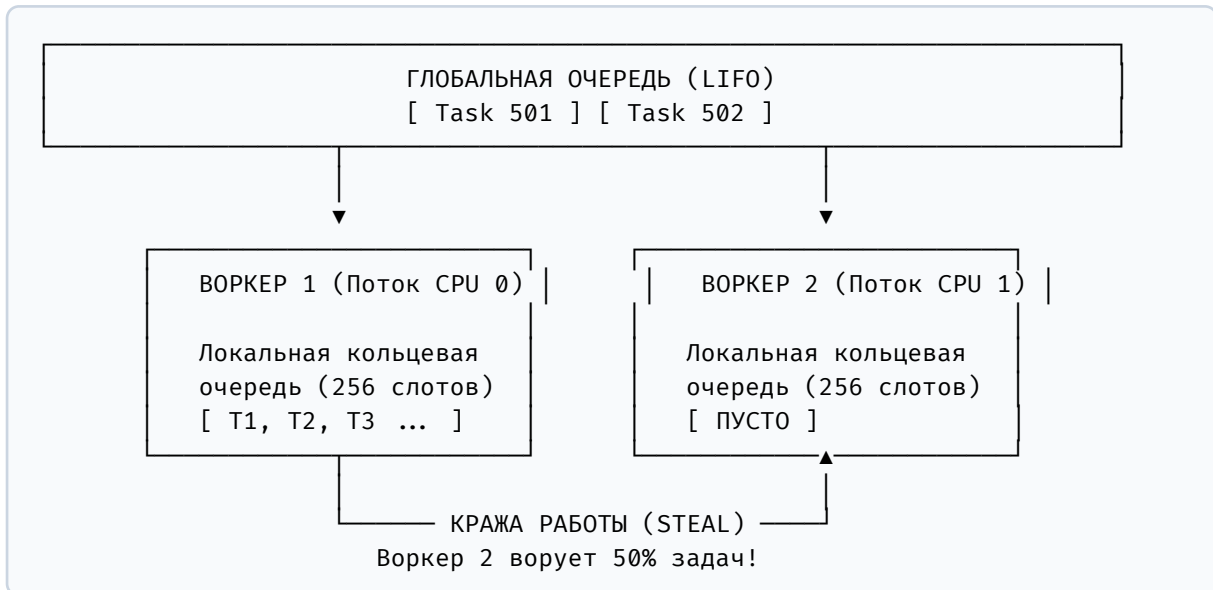
45.020 мс | На этот раз `read()` возвращает байты ответа! `poll()` возвращает `Poll::Ready!`

—

Глава 6. Анатомия рантайма Tokio: Реактор и Планировщик

6.1. Архитектура Multi-Threaded Work-Stealing Scheduler

Рантайм Tokio спроектирован по принципу максимальной локальности кэшей процессора:



1. **Локальная очередь (Local Run Queue):** Каждый рабочий поток имеет кольцевой буфер на **256 задач**. Добавление и извлечение задач в свою очередь поток производит через атомарные операции без захвата глобальных мьютексов (Lock-Free).
2. **Кража работы (Work-Stealing):** Если у Воркера 2 закончились задачи, он не засыпает, а смотрит в очередь Воркера 1. Через атомарную операцию он крадет **половину (например, 128 задач)** из его очереди и переносит к себе.
3. **Борьба с голоданием (Anti-Starvation):** Каждые 61 итерацию рабочий поток принудительно проверяет **Глобальную очередь**, чтобы задачи, попавшие туда, не зависли навечно.

Глава 7. Стримы (stream): Асинхронные потоки данных

7.1. Трейт `Stream` и модель `Pull`

Стрим — это асинхронный аналог `Iterator`. Но если `Iterator::next()` либо сразу возвращает элемент, либо зависает, то `Stream` возвращает результат через механизм `Poll`:

```
pub trait Stream {
    type Item;

    fn poll_next(
        self: Pin<&mut Self>,
        cx: &mut Context<'_>
    ) → Poll<Option<Self::Item>>;
}
```

Построчный анатомический разбор:

- `Poll::Ready(Some(item))` — следующий элемент потока готов.
- `Poll::Ready(None)` — поток полностью исчерпан (конец потока).
- `Poll::Pending` — следующий элемент пока не сформирован (например, пакет из сокета еще не долетел). Задача усыпляется до вызова `Waker`.

Стримы работают по принципу **Pull (тяги по требованию)**: генератор данных не производит новые элементы, пока потребитель не вызовет метод `poll_next()`. Это фундаментальная основа защиты от переполнения памяти (**Backpressure**).

Глава 8. Анатомия `buffer_unordered`: Внутреннее устройство

Вот та самая инструкция из нашего сервиса:

```
let results = stream::iter(sites)
    .map(|site| pinger.ping(&site))
    .buffer_unordered(25)
    .collect()
    .await;
```

Разберем каждую точку этой цепочки на молекулярном уровне.

8.1. Построчный анатомический разбор

```
// 1. stream::iter(sites)
```

Превращает готовый вектор `Vec<Site>` в синхронный стрим, реализующий типаж `Stream`. На каждый вызов `poll_next` он просто берет следующий элемент вектора и возвращает `Poll::Ready(Some(site))`.

```
// 2. .map(|site| pinger.ping(&site))
```

Навешивает адаптер. **Важно:** он еще НЕ запускает сетевой запрос! Он просто говорит: «когда из стрима достанут `site`, примени к нему функцию и верни саму `Future` (ленивую стейт-машину пинга)».

```
// 3. .buffer_unordered(25)
```

Оборачивает стрим футур в управляющую структуру `BufferUnordered`. Задает внутреннее окно параллельности, равное **25**.

```
// 4. .collect().await
```

Вызывает метод сбора всех результатов в единый `Vec<PingResult>`. Именно этот `.await` запускает всю машину в движение!

8.2. Как устроен `FuturesUnordered` внутри?

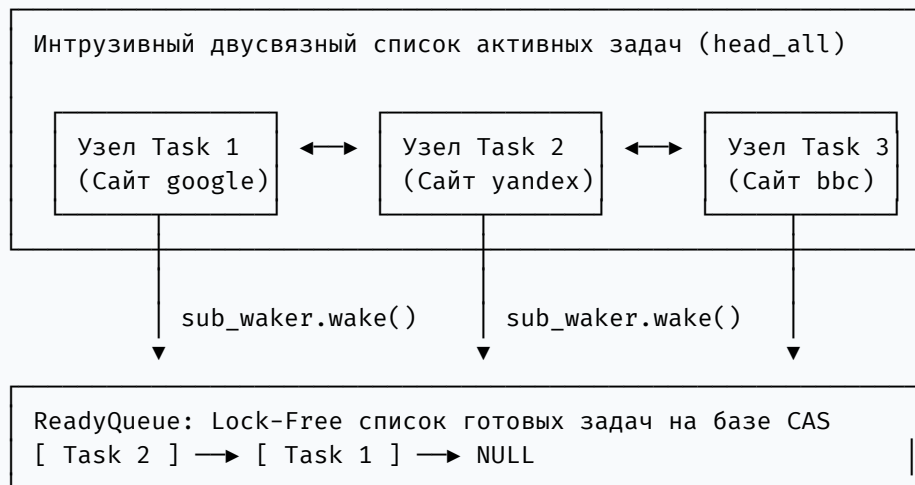
В исходном коде библиотеки `futures-util` структура `FuturesUnordered` устроена следующим образом:

```
pub struct FuturesUnordered<Fut> {
    // Голова интрузивного двусвязного списка всех активных задач
    head_all: *mut Task<Fut>,

    // Lock-Free очередь задач, готовых к опросу прямо сейчас
    ready_to_run_queue: Arc<ReadyToRunQueue<Fut>>,

    // Счетчик запущенных футур (не более 25)
    len: usize,
}
```

СТРУКТУРА ПАМЯТИ FUTURES UNORDERED (25 СЛОТОВ)



⚙️ Уровень гика (Ядро ОС, регистры CPU и структуры памяти):

Почему список называется **интрузивным (intrusive)**? В обычном `std::collections::LinkedList` сам список аллоцирует узлы-обертки `Node { value, prev, next }`. В интрузивном списке указатели `prev` и `next` **встроены прямо внутрь самой структуры задачи Task!** Благодаря этому:

- Нет двойных аллокаций.
- Задача удаляется из середины списка за **1 операцию перезаписи указателей ($O(1)$)** в тот самый момент, когда вернула `Poll::Ready`.

8.3. Механика Sub-Wakers: Почему опрос занимает $O(1)$

Представьте, что у нас крутится 25 сетевых запросов. Если бы `FuturesUnordered` был устроен глупо, при получении сетевого пакета от любого сайта ему пришлось бы обходить **все 25 футур по очереди** и вызывать у них `poll()`. Это была бы катастрофическая потеря процессорного времени ($O(N)$).

Как эта проблема решена:

1. Для каждого из 25 слотов создается свой собственный **Sub-Waker**.
2. В данные этого Sub-Waker вшит сырой указатель на узел конкретной задачи `*const Task<Fut>`.
3. Когда сетевой сокет сайта №2 ловит пакет, вызывается `sub_waker.wake()`.
4. Функция `wake()` не опрашивает все задачи! Она атомарно (через инструкцию процессора `LOCK CMPXCHG / Compare-And-Swap`)¹⁴ помещает адрес узла Задачи 2 в голову очереди `ready_to_run_queue`.
5. Когда Tokio вызывает `poll_next` у `BufferUnordered`, он просто читает один готовый элемент из `ready_to_run_queue` за $O(1)$ и вызывает `poll()` **только у Задачи 2!**

8.4. Почему `buffer_unordered(25)` делает Семафор избыточным?

Посмотрим на код структуры `BufferUnordered`:

¹⁴CMPXCHG (Compare and Exchange) — атомарная инструкция ассемблера x86-64, позволяющая обновить указатель в памяти без блокировки операционной системы и без мьютекса (Lock-Free concurrency).

```
impl<St> Stream for BufferUnordered<St>
where
    St: Stream,
    St::Item: Future,
{
    type Item = <St::Item as Future>::Output;

    fn poll_next(mut self: Pin<&mut Self>, cx: &mut Context<'_>) →
    Poll<Option<Self::Item>> {
        // ЭТАП 1: НАПОЛНЕНИЕ (Пока меньше лимита — достаём новые футуры)
        while self.in_flight.len() < self.limit {
            match self.stream.as_mut().poll_next(cx) {
                Poll::Ready(Some(future)) ⇒ {
                    self.in_flight.push(future); // Засунули в пул (стало на
1 больше)
                }
                Poll::Ready(None) ⇒ break, // Стрим кончился
                Poll::Pending ⇒ break, // Новых элементов пока нет
            }
        }

        // ЭТАП 2: ОПРОС ГОТОВЫХ
        // Опрашивает ReadyQueue пула
        self.in_flight.poll_next_unpin(cx)
    }
}
```

Посмотрите на строчку:

```
while self.in_flight.len() < self.limit
```

Это и есть **железный счетчик семафора**!

- Если в пуле уже крутятся 25 футур, условие цикла становится ложным ($25 < 25 = \text{false}$).
- Цикл прерывается.
- Метод `self.stream.poll_next()` **даже не вызывается**! 26-й сайт из вектора просто остаётся лежать в памяти нетронутым.
- Как только любая футура из 25 завершилась, `self.in_flight.len()` уменьшается до 24.
- При следующем же тике условие $24 < 25$ снова становится истинным, и из стрима мгновенно достаётся 26-я футура!

Добавлять сюда отдельный `tokio::sync::Semaphore` — это всё равно что вешать второй амбарный замок на дверь, которая уже заперта на электронный ригель.

Глава 9. Полный анатомический разбор воркера Uptime-сервиса

Теперь соединим всю теорию с реальным кодом нашего сервиса из файла `src/main.rs`:

```
// =====
// ШАГ 1: Атомарная выборка 500 сайтов из PostgreSQL (Lease Pattern)
// =====
let sites = repo.get_sites_for_ping(500).await?;
if sites.is_empty() {
    continue;
}
```

Построчный анатомический разбор:

- База данных выполняет запрос с `FOR UPDATE SKIP LOCKED`.
- Все 500 строк атомарно помечаются `status = 'processing'`.
- Соединение с БД мгновенно закрывается и возвращается в пул `PgPool`.
- В памяти у нас чистый вектор `Vec<Site>`, транзакция в БД не висит.

```
// =====
// ШАГ 2: Конкурентный параллельный пинг ровно по 25 соединений
// =====
let results: Vec<PingResult> = stream::iter(sites)
    .map(|site| {
        let pinger = pinger.clone();
        async move {
            let site_id = site.id.expect("site id required");
            match pinger.ping(&site).await {
                Ok(resp) => PingResult {
                    site_id,
                    duration_ms: resp.ping_duration.as_secs_f64() * 1000.0,
                    extra:
serde_json::to_value(resp.extra).unwrap_or_default(),
                },
                Err(err) => PingResult {
                    site_id,
                    duration_ms: 0.0,
                    extra: serde_json::json!({ "error": err.to_string() }),
                },
            }
        }
    })
    .buffer_unordered(25) // 🖱 Защита от исчерпания дескрипторов ОС
    .collect()           // 🖱 Ожидание выполнения всех 500 проверок
    .await;
```

Построчный анатомический разбор:

- `pinger.clone()`: копирует легковесную обертку над `reqwest::Client`, который внутри держит `Arc` на общий пул сокетов. Никакие сокет не дублируются.
- `match pinger.ping`: обрабатывает как успешный HTTP-ответ, так и сетевой сбой (таймаут, `DNS error`, `connection refused`). В обоих случаях формируется валидный `PingResult`. Ни один сайт не потеряется!
- `buffer_unordered(25)`: гарантирует, что в операционной системе открыто не более 25 файловых дескрипторов сокетов.

```
// =====
// ШАГ 3: Односекундный сброс батча в базу данных
// =====
repo.save_pings_batch(results).await?;
```

Построчный анатомический разбор:

- Открывается 1 быстрая транзакция `tx`.
- Через `QueryBuilder` формируется 1 большой SQL-запрос `INSERT INTO site_pings VALUES ( ... ), ( ... ), ...`.
- Через `UPDATE sites SET status = 'idle', last_check = NOW() WHERE id = ANY($1)` все 500 сайтов переводятся в режим ожидания следующей проверки.
- `tx.commit().await` фиксирует изменения за 5–10 миллисекунд.

Глава 10. Энциклопедический словарь-справочник

1. Системный уровень (Ядро Linux, C-библиотеки и ОС)

- **Файловый дескриптор (File Descriptor, FD):** Неотрицательное 32-битное целое число, служащее ключом в таблице дескрипторов процесса в ядре Linux. Ассоциировано с файлом, сокетом или каналом (pipe).
- **Системный вызов (Syscall):** Низкоуровневая инструкция процессора (`syscall` в x86-64 / `svc` в ARM), переключающая процессор из непривилегированного режима пользователя (Ring 3) в режим ядра (Ring 0) для выполнения операций с оборудованием.
- **`epoll`:** Высокопроизводительная подсистема ядра Linux для мультиплексирования ввода-вывода. Отслеживает события на тысячах файловых дескрипторов за константное время $O(1)$.
- **`O_NONBLOCK`:** Флаг дескриптора файла, отключающий перевод вызывающего потока в режим ожидания (Sleep/Blocked). Вместо сна ядро мгновенно возвращает ошибку `EAGAIN` / `EWOULDBLOCK`.
- **Контекстное переключение (Context Switch):** Процедура сохранения регистров текущего потока в оперативную память, сброса кэша ассоциативной трансляции страниц памяти (TLB Flush) и загрузки регистров другого потока.
- **Аппаратное прерывание (Hardware Interrupt, IRQ):** Электрический сигнал от внешнего устройства (сетевая карта, диск) к контроллеру прерываний процессора (APIC), заставляющий CPU временно приостановить поток и запустить обработчик прерывания ядра.
- **DMA (Direct Memory Access):** Технология прямого доступа к оперативной памяти без участия CPU. Позволяет сетевой карте записывать входящие пакеты напрямую в RAM.

2. Уровень архитектуры компилятора Rust

- **Future (Футура):** Трейт, представляющий отложенное асинхронное вычисление, опрашиваемое через метод `poll`.
- **Стейт-машина (State Machine):** Автомат состояний в виде скрытого `enum`, генерируемый компилятором `rustc` из каждого блока `async fn`.
- **`Pin<&mut T>`:** Умный указатель, запрещающий перемещение значения `T` по новому адресу в памяти. Необходим для самоссылающихся структур (где поле указывает на другое поле того же объекта).
- **Waker:** Обертка над сырым указателем и виртуальной таблицей `RawWakerVTable`. Позволяет любой подсистеме уведомить планировщик о готовности задачи к возобновлению работы.
- **Кооперативная многозадачность (Cooperative Multitasking):** Модель параллелизма, при которой задачи работают непрерывно и отдают процессор добровольно только в определенных точках (в Rust — оператор `.await`).

3. Уровень рантайма Tokio и алгоритмов

- **Executor (Исполнитель):** Подсистема рантайма Tokio, управляющая пулом рабочих системных потоков и распределением задач.
- **Reactor (Реактор):** Подсистема рантайма Tokio, непрерывно выполняющая вызов `epoll_wait` и связывающая системные сетевые события со структурами `Waker`.
- **Work-Stealing:** Алгоритм планирования, при котором рабочий поток, оставшийся без задач в локальной очереди, атомарно крадет половину задач у другого загруженного потока.
- **FuturesUnordered:** Интрузивная двусвязная структура данных, позволяющая конкурентно выполнять произвольное число футур и возвращать их результаты по мере готовности за $O(1)$.

- **Sub-Waker:** Специализированный вейкер внутри `FuturesUnordered`, содержащий прямой указатель на конкретный узел задачи, что исключает необходимость перебора всех активных футур при пробуждении.
- **Backpressure (Обратное давление):** Механизм координации скорости между источником данных и потребителем, предотвращающий неограниченный рост буферов в оперативной памяти.
- **CAS (Compare-And-Swap):** Атомарная инструкция процессора, позволяющая обновить значение переменной в памяти только в том случае, если текущее значение совпадает с ожидаемым. Фундамент неблокирующих очередей (Lock-Free).

Поздравляем! Теперь вы обладаете полным пониманием асинхронной архитектуры Rust на всех уровнях абстракции. 🦀🚀